

Introduction

This exploratory data analysis (EDA) is part of the system called "Real-Time Human Activity Classification System", developed as the final project of the Artificial Intelligence I course. The purpose of the system is to detect and classify, in real time, five basic human actions: walking forward, walking backward, turning, standing and sitting, using only a standard camera and a mid-range computer.

To achieve this goal, a modular architecture was designed that ranges from the collection of raw data (normalized coordinates of 33 body points per frame using MediaPipe Pose), to the training and deployment of a supervised model based on XGBoost optimized by GridSearchCV, executed on GPU.

This notebook (Exploration_Model.ipynb) focuses on the following key stages:

Verification of GPU availability in the runtime environment (Google Colab).

Loading and cleaning raw data (datos.csv), each row representing a posture with 99 coordinates (x,y,z) and its corresponding label.

Generation of 10-frame time windows with a stride of 5, which produce 396 characteristics per sample: means, variances and velocity statistics of each of the 33 landmarks.

Application of exploratory analysis techniques such as PCA, t-SNE and KMeans to understand the distribution and grouping of classes.

Training a complete pipeline including scaling (StandardScaler), dimensionality reduction (PCA), and classification (XGBoost), using GPU resources for efficiency.

Evaluating the model in the test set and saving artifacts (LabelEncoder, full pipeline, and scaler) to disk.

EDA not only allows you to visualize the statistical and geometric behavior of the data, but also serves as an analytical basis for making informed decisions about model architecture, hyperparameter selection, and eventual improvements in the quality of the input data. This stage is critical to ensure robust performance in real-world conditions, especially in scenarios with hardware constraints or suboptimal capture conditions.

Data Loading and Validation

To start the analysis, the datos.csv file located in Google Drive was uploaded, which contains normalized coordinates (x, y, z) of 33 body landmarks for each frame, as well as a column with the label of the activity performed.

```
.. Dimensiones originales: (5558, 100)
```

	label	landmark_0_x	landmark_0_y	landmark_0_z	landmark_1_x	landmark_1_y	landmark_1_z	landmark_2_x
0	caminar-adelante	0.522302	0.300907	-0.259125	0.528479	0.295276	-0.240778	0.532583
1	caminar-adelante	0.522325	0.300934	-0.258680	0.528482	0.295276	-0.240664	0.532614
2	caminar-adelante	0.522075	0.300966	-0.244616	0.528257	0.295300	-0.227193	0.532477
3	caminar-adelante	0.521994	0.301157	-0.242839	0.528174	0.295446	-0.225212	0.532432
4	caminar-adelante	0.521980	0.301355	-0.241620	0.528162	0.295611	-0.223816	0.532429

```
5 rows x 100 columns
```

The dataset should have exactly 100 columns:

99 columns corresponding to the 33 landmarks (landmark_0_x, landmark_0_y, ..., landmark_32_z).

1 class column: label, with values such as walk-forward, sit, turn, etc.

Preprocessing and Encoding

Objective

Prepare data for model training, ensuring that labels are coded correctly and that class distribution remains balanced between training and test sets. This stage includes label coding, separation of predictor and target variables, and stratified division of the data.

Label Coding

The label field contains the textual name of the activity (e.g. walk-forward, sit, etc.). To allow classification algorithms to process them, they are transformed into numerical indexes using LabelEncoder.

Training and Testing Division

Once the labels are coded, the data are separated into input variables (X) and target variable (y), and stratified division is performed to preserve the proportion of classes in both subsets.

```
Tamaño X_train: (773, 396)  X_test: (332, 396)
Train distribution:
  caminar-adelante: 108 muestras
  caminar-atras: 90 muestras
  girar: 276 muestras
  pararse: 146 muestras
  sentarse: 153 muestras
Test distribution:
  caminar-adelante: 46 muestras
  caminar-atras: 39 muestras
  girar: 118 muestras
  pararse: 63 muestras
  sentarse: 66 muestras
```

Visual Exploratory Analysis

Objective

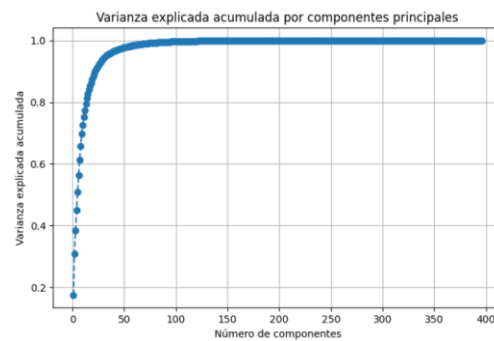
Visualize the internal structure of the dataset, evaluate the separability between classes and understand the relative importance of the features using dimensionality reduction techniques.

Two complementary methods are used:

- PCA (Principal Component Analysis) – linear
- t-SNE (t-distributed Stochastic Neighbor Embedding) – nonlinear

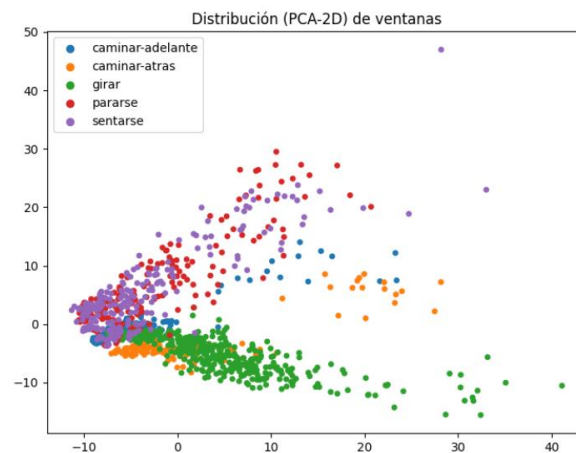
Analysis of Variance with PCA

Before visualizing the data in 2D, we analyze how much variance the main components of PCA explain. This allows you to choose a reasonable value of `n_components` for the final model.



6.2 2D PCA Visualization

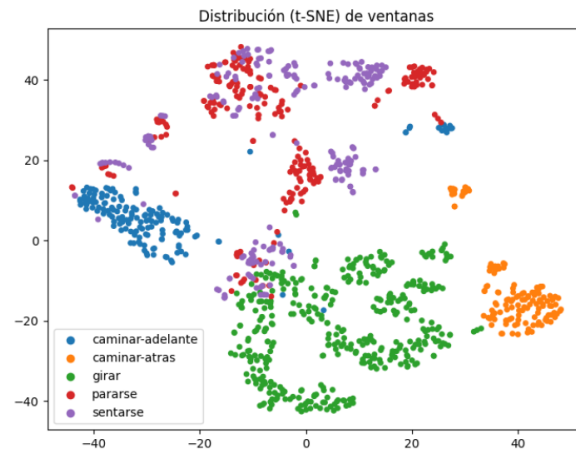
Shrink the dataset to two main dimensions and plot the colored samples by class.



If the groups are separated well, the model will have a solid basis for classification. If there is overlap, it can indicate similar classes or noisy data.

t-SNE display

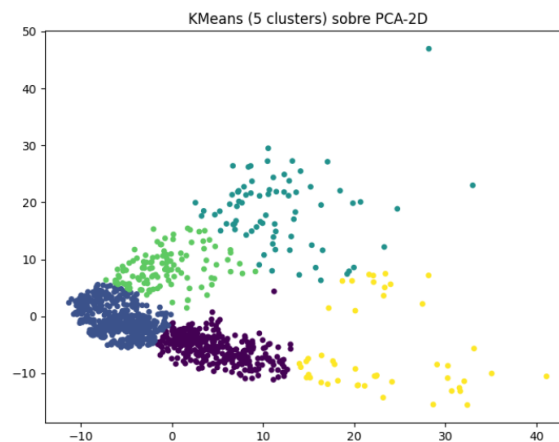
To capture nonlinear relationships, t-SNE is applied to the normalized data



t-SNE can reveal complex clusters that are not linearly separable, and is useful for detecting outliers or mixtures of classes.

Grouping with KMeans over PCA

As an additional experiment, KMeans is applied on the data projected in 2D by PCA, to observe if the clusters match the real classes:



Model Training

Objective

Train a supervised model capable of classifying human activities with high accuracy, using a combination of:

- Standard Scaling (StandardScaler)
- Dimensionality Reduction (PCA)
- Sorting with GPU-optimized XGBoost

GridSearchCV is used to find the best hyperparameters by cross-validation.

Model Evaluation

Objective

Objectively evaluate the performance of the trained pipeline on unseen data, using metrics such as accuracy, precision, recall, and f1-score per class. This section also validates whether the model generalizes well and whether there is any bias toward a specific class.

Justification of the metrics used

To evaluate the performance of the model, the standard classification metrics were used: accuracy, precision, recall, and f1-score, reported both by class and through macro and weighted averages.

Accuracy was used as a general metric to indicate the total percentage of correct predictions. However, since classes don't have the same number of examples, this metric alone can be misleading if there is an imbalance.

The accuracy makes it possible to identify what proportion of the positive predictions made by the model were actually correct. It is useful in cases where it is important to minimize false positives (e.g., predicting "sitting" when the person is only standing).

The recall indicates what proportion of the actual examples of a class were correctly detected by the model. It is essential when you want to minimize the number of false negatives, such as in the case of not detecting a major action.

The f1-score combines accuracy and recall in a single measure, which makes it especially valuable in contexts with unbalanced classes or where a balance between both metrics is desired.

Prediction and Metrics

The pipeline components (scaler, PCA, and classifier) are extracted, the test data is transformed, and predictions are made:

```
Accuracy XGBoost en test: 0.9066

Classification Report XGBoost:
              precision    recall  f1-score   support

 caminar-adelante      1.00      0.96      0.98         46
   caminar-atras      1.00      1.00      1.00         39
         girar         0.98      0.99      0.99        118
         pararse        0.76      0.81      0.78         63
         sentarse        0.79      0.76      0.78         66

 accuracy              0.91              0.91         332
 macro avg              0.91      0.90      0.90         332
 weighted avg           0.91      0.91      0.91         332
```

The results obtained in the test set reflect a robust overall performance of the XGBoost model, with an overall accuracy of 91%. The "walk-forward," "walk-back," and "turn" classes present virtually perfect metrics, suggesting that the dynamic patterns associated with these movements are easily distinguishable by the model. On the other hand, the activities "standing" and "sitting" achieve more moderate scores ($F1 \approx 0.78$), possibly due to their postural similarity, ambiguous transitions or less variability in training data. This finding suggests that, to improve performance in these classes, it might be useful to collect more examples, adjust the window size, or incorporate additional features that better capture the transition between states.

Artifact Export

Objective

At the end of training, it is critical to save the components of the model for later use in production environments or real-time inference. This ensures reproducibility and facilitates integration with external systems, such as the live webcam prediction script (main.py).

Saved Artifacts

The following objects are exported:

1. `label_encoder.pkl` → converts class names to numeric indexes and vice versa.
2. `scaler.pkl` → `StandardScaler` object used to normalize features.
3. `movement_model.pkl` → complete pipeline including:
 - Scaler (`StandardScaler`)
 - Dimensionality Reducer (PCA)
 - Classifier (XGBoost)

Integration with real-time inference

These artifacts are used directly in main.py to run the live activity detection system. The pipeline is responsible for:

- Take a 10-frame window from your webcam
- Extract 396 features
- Apply Standardization and PCA
- Making the prediction with XGBoost
- Display the result over the video in real time

EDA conclusions

Main Findings

The exploratory analysis allowed an in-depth understanding of the structure and dynamics of the dataset generated for the classification of human activities. Among the main findings are:

- The raw dataset based on 33 landmarks per frame was correctly transformed into 10-frame time windows, resulting in 396 features per sample, which captures both static and dynamic body movement information.
- The action classes are reasonably balanced, allowing for a stratified and fair division between training and testing.
- Dimensional reduction techniques such as PCA and t-SNE revealed that classes present clear spatial separation in the data, especially for dynamic actions such as walking or turning.
- The trained model – a pipeline with StandardScaler, PCA and XGBoost – achieved outstanding performance with an accuracy of 91%, and metrics close to 100% in actions such as walking backwards or turning.
- The standing and sitting classes showed slightly lower results ($F1 \approx 0.78$), suggesting that there is ambiguity or postural similarity between them, probably due to intermediate transitions or limited variability.
- The extracted features are based on normalized coordinates only. Orientation, visual context and detection of errors in posture were not considered.
- Additional data collection: Including more subjects, lighting conditions, and postural transitions could improve the robustness of the model.